DESENVOLVIMENTO DE SOFTWARE II	
ALUNO: Virginaldo Eraldo de Araujo	**Nota:**
Prof: Tassio Valle	**Data:** 24/10/2026
TRABALHO 4 – PROJETO FINAL: APLICAÇÃO EM TEMPO REAL CONTAINERIZADA TRILHA 4	

Projeto escolhido: Sala de Aula Virtual em Tempo Real

Tecnologias: Node.js, Express, React, Socket.IO, Prisma ORM, PostgreSQL e Docker.

1. Introdução

O objetivo do presente trabalho é desenvolver uma aplicação web completa utilizando os conhecimentos adquiridos na disciplina de desenvolverde Software I e II . O projeto conta com API REST, banco de dados relacional, arquitetura em camadas, comunicação em tempo real com WebSockets e containerização com Docker.

A aplicação selecionada é uma Sala de Aula Virtual em Tempo Real chamada EduConnect, que visa facilitar a comunicação entre professores e alunos, permitindo a realização de atividades interativas e o monitoramento em tempo real da participação em sala de aula.

A proposta considera a realidade das escolas públicas, nas quais as ferramentas digitais podem contribuir para tornar as aulas mais participativas, organizadas e dinâmicas.

2. Objetivo geral

Criar uma aplicação web de ensino que possibilita a comunicação em tempo real entre docentes e discentes, empregando as mais recentes tecnologias de desenvolvimento de software, com organização, persistência de dados, segurança básica e facilidade de implantação.

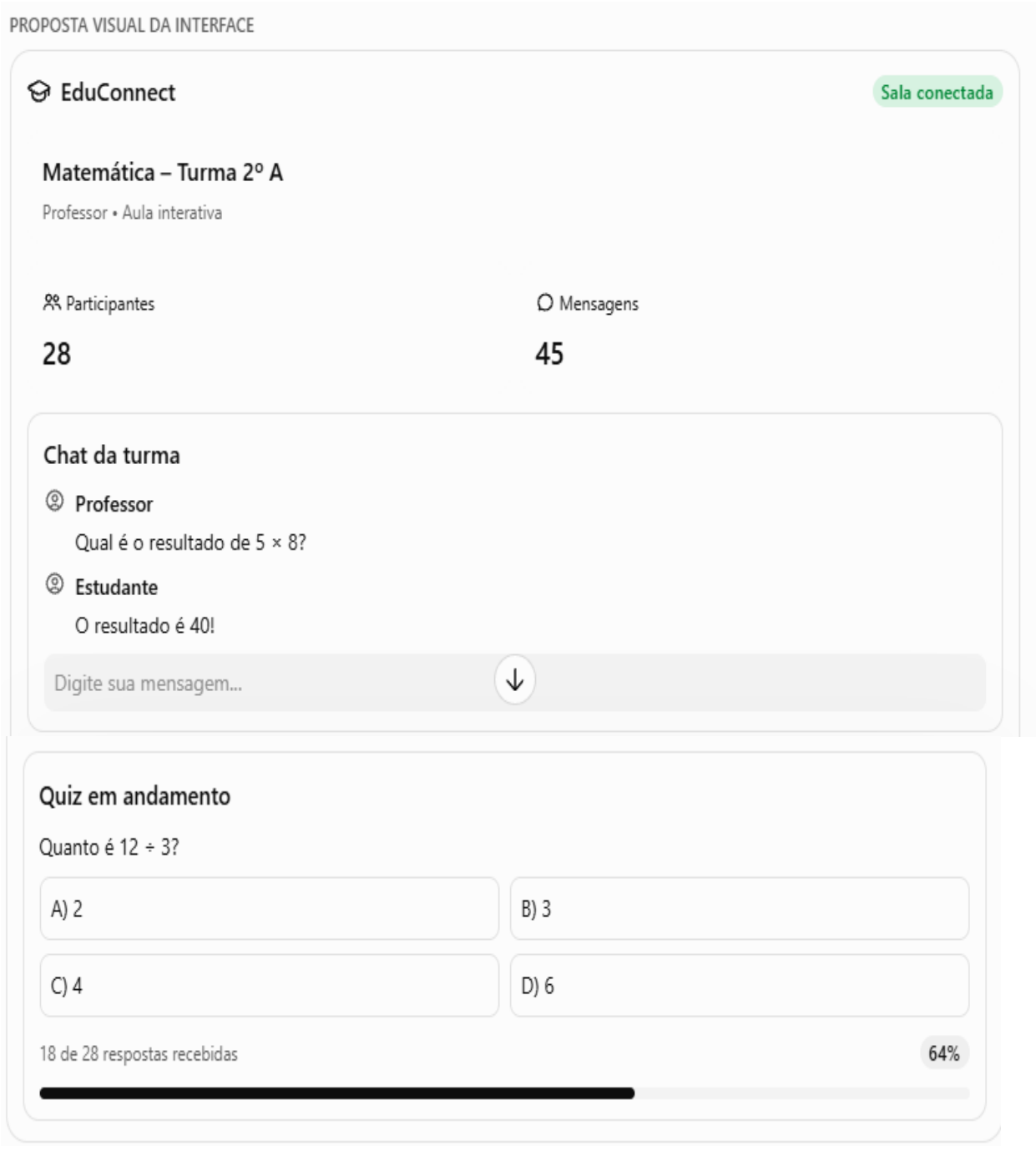
Objetivos específicos

- Criar uma API REST documentada para gerenciar salas, participantes, mensagens e atividades.
- Implementar comunicação em tempo real utilizando Socket.IO.
- Armazenar informações em um banco PostgreSQL por meio do Prisma ORM.
- Organizar o código em camadas independentes.
- Desenvolver uma interface responsiva em React.
- Utilizar Docker e Docker Compose para executar a aplicação.

- Implementar validações, tratamento de erros e testes.
- Documentar o funcionamento e a arquitetura do sistema.

3. Descrição e funcionamento da aplicação

O EduConnect permitirá que professores criem salas virtuais e que estudantes participem utilizando seu nome e a identificação da turma, sem necessidade de cadastro com senha.



4. Funcionalidades obrigatórias

O sistema será organizado em seis funcionalidades principais.

1. Identificação dos usuários: professores e estudantes informarão nome e papel ao entrar na sala. A identificação não exigirá login.
2. Criação de salas: o professor poderá criar uma sala para cada turma, com código de acesso.
3. Chat em tempo real: as mensagens serão exibidas imediatamente para todos os participantes conectados e armazenadas no banco de dados.
4. Lista de participantes: o sistema mostrará os usuários conectados, atualizando a lista automaticamente.
5. Quiz interativo: o professor poderá enviar perguntas de múltipla escolha, e os estudantes responderão em tempo real.
6. Painel de resultados: o professor acompanhará a quantidade de respostas e a distribuição das alternativas escolhidas, sem atualizar a página.

5. Tecnologias utilizadas

Tecnologia	Finalidade
Node.js	Executar o servidor
Express	Construir a API REST
React	Desenvolver a interface
Socket.IO	Comunicação em tempo real
Prisma ORM	Acesso e persistência dos dados
PostgreSQL	Banco de dados relacional
Zod	Validação dos dados recebidos
Swagger/OpenAPI	Documentação da API
Docker	Empacotar a aplicação
Docker Compose	Orquestrar aplicação e banco
Git/GitHub	Versionamento e colaboração

6. Arquitetura do sistema

Será utilizada uma arquitetura em camadas, mantendo as regras de negócio separadas das tecnologias de infraestrutura.



Serão aplicados três padrões principais:

- Repository: separa as operações de banco de dados das regras de negócio.
- Factory: centraliza a criação dos serviços e suas dependências.
- Injeção de Dependência: permite que os serviços recebam repositórios por interfaces ou contratos, facilitando testes e manutenção.

7. Modelagem do banco de dados

O sistema utilizará seis tabelas principais, com relacionamentos entre elas.

Tabela	Informações armazenadas
Usuario	ID, nome e papel
Sala	ID, título e código
Participacao	Usuário e sala vinculados
Mensagem	Autor, sala, conteúdo e data
Quiz	Pergunta, alternativas e resposta correta
Resposta	Estudante, quiz, alternativa e horário

Os relacionamentos vão possibilitar que uma sala tenha múltiplos participantes, mensagens e quizzes. Cada questionário pode ser respondido por vários alunos.

Para que preserve a integridade, cada participante poderá responder apenas uma vez a cada questionário, e as operações que serão realizadas em conjunto terão suas transações restritas.

8. API REST

A API terá pelo menos quatro recursos com operações completas de criação, consulta, atualização e exclusão.

Método	Endpoint	Finalidade
GET / POST	`/api/usuarios`	Listar e cadastrar usuários
GET / PUT / DELETE	`/api/usuarios/:id`	Consultar, editar e excluir
GET / POST	`/api/salas`	Listar e criar salas
GET / PUT / DELETE	`/api/salas/:id`	Gerenciar sala específica
GET / POST	`/api/mensagens`	Consultar e enviar mensagens
GET / PUT / DELETE	`/api/mensagens/:id`	Gerenciar mensagens
GET / POST	`/api/quizzes`	Listar e criar quizzes
GET / PUT / DELETE	`/api/quizzes/:id`	Gerenciar quizzes
GET / POST	`/api/respostas`	Consultar e registrar respostas
GET / PUT / DELETE	`/api/respostas/:id`	Gerenciar respostas

As listagens utilizarão paginação e filtros, por exemplo:
`/api/mensagens?salaId=1&page=1&limit=20`.

A documentação estará disponível em `/docs`, e os erros serão apresentados em um formato JSON padronizado. As permissões de alteração serão verificadas no servidor, respeitando as limitações de uma identificação sem autenticação.

9. Comunicação em tempo real com WebSockets

A comunicação em tempo real será implementada com Socket.IO. Cada turma terá sua própria sala de comunicação, impedindo que mensagens de uma turma sejam enviadas para outra.

Evento	Função
`sala:entrar`	Participante entra na sala
`sala:participantes`	Atualiza a lista de conectados
`chat:enviar`	Envia uma mensagem
`chat:novaMensagem`	Distribui a mensagem recebida
`quiz:criar`	Professor publica uma pergunta
`quiz:responder`	Estudante envia uma resposta
`quiz:resultado`	Atualiza o painel de resultados
`sala:sair`	Participante deixa a sala

Exemplo de mensagem enviada pelo estudante:

```
{
  "salaId": 1,
  "usuarioId": 12,
  "conteudo": "Professor, pode explicar novamente?"
}
```

O servidor validará os dados, verificará se o participante pertence à sala e armazenará a mensagem antes de distribuí-la.

Em caso de queda da conexão, o cliente tentará se reconectar automaticamente. Após a reconexão, consultará novamente o histórico, a lista de participantes e o quiz ativo para recuperar o estado atualizado.

10. Containerização com Docker

O aplicativo será configurado para rodar em qualquer computador com Docker, sem que seja necessário instalar o Node.js ou PostgreSQL manualmente.

O Docker Compose irá gerenciar dois serviços principais: app, que cuida do backend e da interface compilada, e db, que é o serviço do PostgreSQL.

O banco terá um volume constante, e o serviço do aplicativo esperará que o banco fique saudável para começar. O Dockerfile usará uma construção de várias etapas para criar uma imagem menor e será executada por um usuário sem privilégios.

O endpoint `/health` permitirá verificar o funcionamento do servidor.

O arquivo `.env.example` conterá apenas valores de exemplo, sem credenciais reais de produção.

O comando previsto para iniciar a aplicação será:

```
cp .env.example .env
docker compose up -d --build
```

As migrations e o seed poderão ser executados dentro do container:

```
docker compose exec app npx prisma migrate deploy
docker compose exec app npm run seed
```

A interface, a API e o WebSocket serão disponibilizados na mesma porta do backend, simplificando o acesso.

11. Organização dos arquivos

```
trabalho4/
├── backend/
│   ├── src/
│   │   ├── domain/
│   │   ├── application/
│   │   ├── adapters/
│   │   │   ├── http/
│   │   │   ├── websocket/
│   │   │   └── persistence/
│   │   ├── config/
│   │   └── server.js
│   ├── prisma/
│   │   ├── migrations/
│   │   ├── schema.prisma
│   │   └── seed.js
│   ├── tests/
│   ├── Dockerfile
│   └── .dockerignore
├── frontend/
│   ├── src/
│   └── package.json
├── docker-compose.yml
├── .env.example
├── .gitignore
├── ARQUITETURA.md
└── README.md
```

12. Testes e validação

Serão realizados testes automatizados e manuais para garantir a qualidade da aplicação. Os testes da API vão verificar operações CRUD, paginação, filtragem e tratamento de erros. Os testes de persistência irão verificar relacionamentos e transações. Serão feitos testes de WebSocket com, no mínimo, dois navegadores conectados à mesma sala. Será feito um um teste para ver se as mensagens, os participantes e os resultados feitos dos quizzes são atualizados ao mesmo tempo.

Serão feitos também testes de desconexão e reconexão, envio de dados inválidos e tentativa de acesso a outras salas.

Finalmente, a aplicação deverá ser testada com uma cópia limpa do repositório para garantir que o README contenha instruções suficientes para iniciar todos os serviços.

13. Organização da equipe

A equipe poderá ser formada por três integrantes, com divisão de responsabilidades e participação conjunta nas decisões.

Integrante	Responsabilidades principais
Integrante 1	API REST, regras de negócio e arquitetura
Integrante 2	Banco de dados, ORM, WebSockets e testes
Integrante 3	Interface React, Docker e documentação

Todos deverão contribuir com o GitHub, revisar partes do código e participar dos testes e da apresentação.

14. Resultados esperados

O objetivo é criar uma aplicação que possibilite uma comunicação mais eficiente entre professores e alunos, com a realização de atividades interativas em tempo real.

Em termos técnicos, o projeto precisa evidenciar a integração da interface web, API REST, banco de dados, arquitetura em camadas e comunicação em tempo real.

Outro objetivo é que é simples de implementar, possibilitando que a aplicação rode com Docker Compose em diversos computadores.

15. Desafios e possíveis soluções

Um dos grandes desafios garantirão que todos os participantes vejam as mesmas informações simultaneamente. Para garantir isso, os eventos serão estruturados por sala, e caberá ao servidor validar e distribuir as atualizações.

Para prevenir a infecção de registros duplicados, serão rompidas restrições no banco de dados e transações. A solução consistirá em empregar nomes de serviços do Docker Compose, variáveis de ambiente e verificações de saúde. Outro desafio será a manutenção das respostas dos quizzes. prevenir a inserção de registros duplicados, serão implementadas restrições no banco de dados e transações.

A conteinerização também pode trazer desafios, especialmente no que diz respeito à comunicação entre os serviços. A solução consistirá em empregar nomes de serviços do Docker Compose, variáveis de ambiente e checagens de saúde.

16. Conclusão

O projeto EduConnect nos oferece a oportunidade de aplicar tudo o que foi aprendido na disciplina de Desenvolvimento de Software, unindo diversas tecnologias em uma única aplicação.

A proposta deixa claro que o desenvolvimento de sistemas vai muito além de simplesmente criar códigos que funcionem individualmente; é um processo que exige planejamento, organização, interação entre os componentes, segurança, testes e documentação.

A utilização de WebSockets, será possível entender como as informações são compartilhadas em tempo real entre os usuários, enquanto o ORM e a arquitetura em camadas ajudam a manter o sistema organizado e fácil de gerenciar.

Além disso, o Docker permite que a aplicação seja realizada em diversos ambientes, tornando o trabalho acadêmico mais alinhado com as práticas de desenvolvimento de software no mercado profissional.

Conclui-se que o projeto pode auxiliar tanto na capacitação técnica dos membros quanto à construção de um recurso didático que promova a participação e o engajamento dos alunos nas aulas.

.

.